ENME302 Python Refresher Lab – Week 1

This lab is largely a repeat of one of the early labs completed last year in EMTH271 and also repeating some of the concept that you learned in prior courses. The purpose of this lab is solely to re-familiarise yourself with Python and to have a chance to use some basic functions that will be helpful for coding within ENME302. If you are already comfortable with Python, you might not need to complete these tasks.

Task 1 – some basic definitions and manipulations of numpy arrays:

Generate the following matrix as a numpy array:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Create a variable B by defining "B = A", and then change the entry in the middle row and column of B to have a value of 11. Print both A and B.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 11 & 6 \\ 7 & 8 & 9 \end{bmatrix} \qquad B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 11 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

When using Python, the command "B = A" creates a linkage between the two variables, such that changes to B will also change A. This underlying linkage is why both A and B have changed above.

Now change the middle entry in A and B back to 5, and then make a new variable C that is a copy of A, using the command "C = A.copy()". Now change the value in the middle row and column of C to have a value of 15. Print both A and C.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \qquad C = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 15 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Now transpose A and assign that as a new variable called D.

$$D = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$

Generate a new variable E, which is the following 6x6 matrix made up of three smaller 3x3 matrices concatenated together, represented as a numpy array:

$$E = \begin{bmatrix} A_{3x3} & 0_{3x3} \\ 0_{3x3} & A_{3x3} \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 0 & 0 & 0 \\ 4 & 5 & 6 & 0 & 0 & 0 \\ 7 & 8 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 2 & 3 \\ 0 & 0 & 0 & 4 & 5 & 6 \\ 0 & 0 & 0 & 7 & 8 & 9 \end{bmatrix}$$

Undertake a matrix multiplication between variables C and D and assign the result as F:

$$F = C \times D = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 15 & 6 \\ 7 & 8 & 9 \end{bmatrix} \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix} = \begin{bmatrix} 14 & 32 & 50 \\ 52 & 127 & 202 \\ 50 & 122 & 194 \end{bmatrix}$$

Note that this result for F above is a full matrix multiplication, not an element-by-element multiplication.

If we have the following set of simultaneous equations, solve for the unknown variables in the x vector:

System of Equations	Solution
$\begin{bmatrix}28 & 16 & 96\\ 68 & 12 & 34\\ 66 & 50 & 59\end{bmatrix}\begin{Bmatrix}x_1\\ x_2\\ x_3\end{Bmatrix}=\begin{Bmatrix}42\\ 71\\ 80\end{Bmatrix}$	$\begin{Bmatrix}x_1\\ x_2\\ x_3\end{Bmatrix}=\begin{Bmatrix}0.9443\\ 0.2021\\ 0.1284\end{Bmatrix}$

Task 2 – Solve a simple equation:

The aim is to solve the equation: $2\sin(\sqrt{x}) - x = 0$

We can set this equation up in an iterative form, where we begin with an initial guess for x, and then revise the guess on successive iteration. To do this, we can reformulate the equation as:

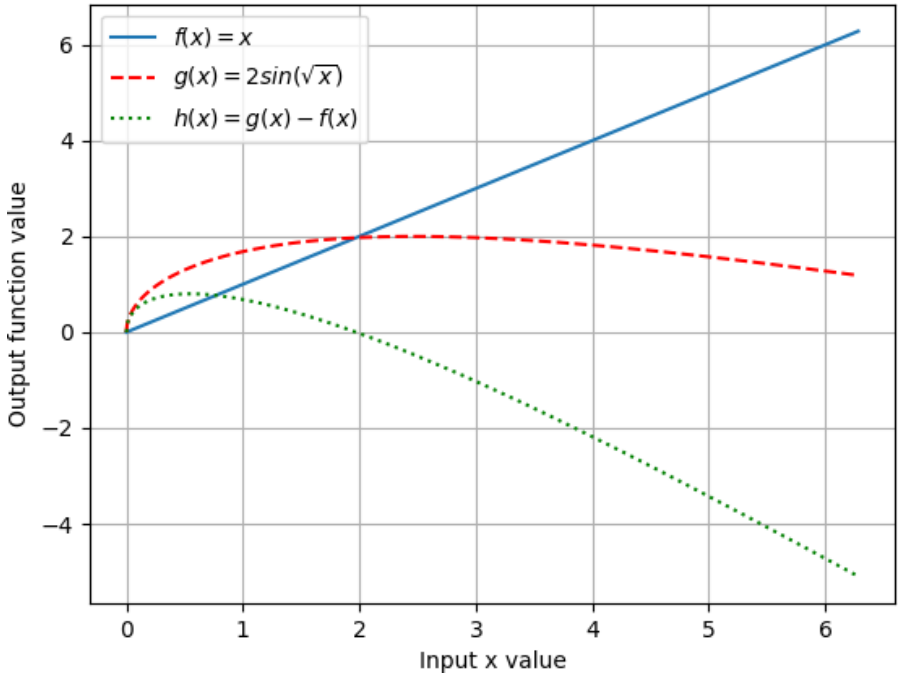
$$x_{i+1} = g(x_i)$$

Where the function $g(x)$ is defined:

$$g(x) = 2\sin(\sqrt{x})$$

To understand the problem before we solve the equation, we will first plot functions of $f(x) = x$, $g(x) = 2\sin(\sqrt{x})$, and $h(x) = g(x) - f(x) = 2\sin(\sqrt{x}) - x$. The point where $h(x)$ crosses the horizontal axis is solution that we are seeking.

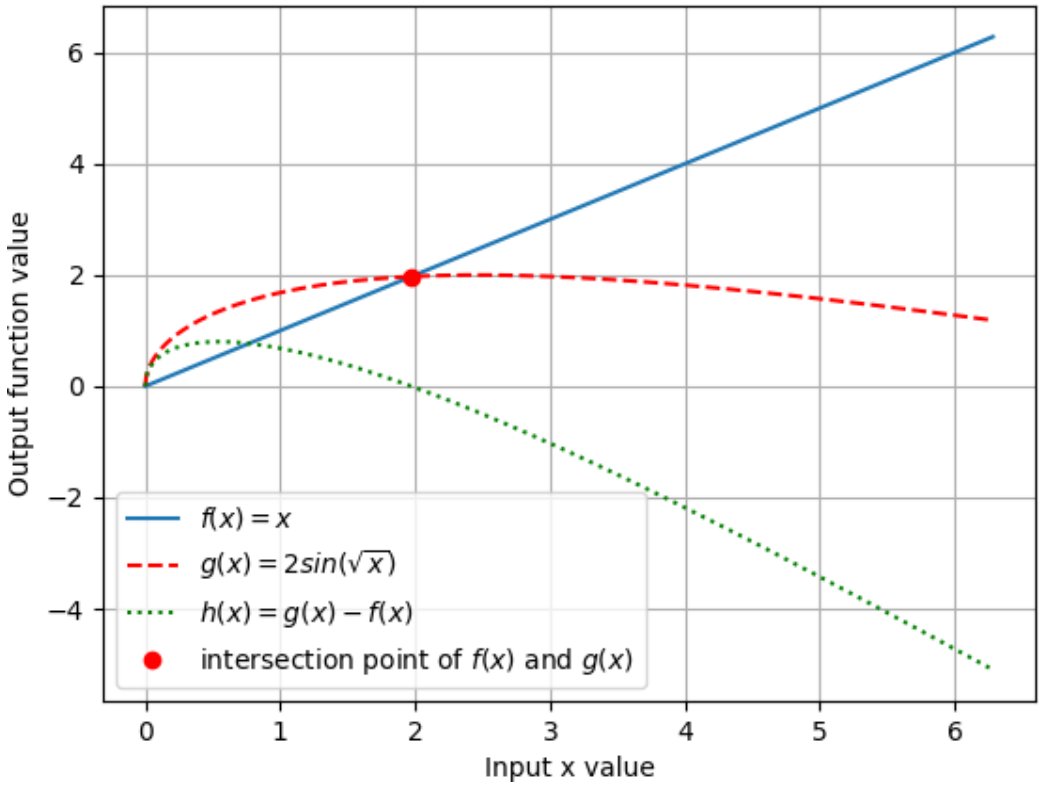
Produce the following plot (you do not necessarily need to be so detailed in the legend labels):



Based upon an initial guess of $x_0 = 0.75$, write some code to generate the successive approximations of x over 10 iterations:

```
The new value of x after 1 iterations is 1.5235199628325784
The new value of x after 2 iterations is 1.8878408918680574
The new value of x after 3 iterations is 1.9613910415195581
The new value of x after 4 iterations is 1.9710680664208446
The new value of x after 5 iterations is 1.972225930003675
The new value of x after 6 iterations is 1.9723627084342208
The new value of x after 7 iterations is 1.9723788412805028
The new value of x after 8 iterations is 1.9723807437844687
The new value of x after 9 iterations is 1.9723809681369364
The new value of x after 10 iterations is 1.9723809945935975
```

Now that you have a value of the intersection point, add a red circle at the point where $f(x)$ and $g(x)$ intersect:



Entry of variable values with large numbers:

Throughout this part of the course, we will regularly be entering numbers that are quite large. For example, the elastic modulus of steel is generally quoted as 200 GPa. We always enter values in their base unit, so this will be 200,000,000,000 Pa. We could enter this as:

```
E = 200*10**9 # Elastic modulus in Pa
```

However, a much easier format is to simply use scientific notation:

```
E = 200e9 # Elastic modulus in Pa
```

Formatting output to display within the Python Shell.

Depending upon the IDE that you typically use, there are many different methods of viewing information in variables. Some IDEs have very helpful methods to visualize the content of different variables, such as the Variable Explorer within Spyder. However, many people will simply print output to the Python Shell.

Based upon the variables used in this lab session, we could simply print output using:

```
print(A)
print(B)
print(C)
print(D)
print(E)
print(F)
```

which would produce the following content within the Python Shell:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
[[ 1  2  3]
 [ 4 11  6]
 [ 7  8  9]]
[[1 2 3]
 [4 5 6]
 [7 8 9]]
[[1 4 7]
 [2 5 8]
 [3 6 9]]
[[1. 2. 3. 0. 0. 0.]
 [4. 5. 6. 0. 0. 0.]
 [7. 8. 9. 0. 0. 0.]
 [0. 0. 0. 1. 2. 3.]
 [0. 0. 0. 4. 5. 6.]
 [0. 0. 0. 7. 8. 9.]]
[[ 14  32  50]
 [ 32  77 122]
 [ 50 122 194]]
```

Note that variables run together and it can be very hard to tell where one variable ends and the other begins. Interpreting and understanding this output can be really challenging. Conversely, tidy formatting can be incredibly important to our ability to visualize and understand the content of variables. If we were to instead use a different format for our print commands, we will get a much clearer output that will be easier to read and support much clearer debugging of issues within our code.

```
print(f"A = \n{A}\n") # the \n command represents a new line and breaks up the variables.
print(f"B = \n{B}\n")
print(f"C = \n{C}\n")
print(f"D = \n{D}\n")
print(f"E = \n{E}\n")
print(f"F = \n{F}\n")
```

```
A =
[[1 2 3]
 [4 5 6]
 [7 8 9]]

B =
[[ 1  2  3]
 [ 4 11  6]
 [ 7  8  9]]
```

```
C =
[[1 2 3]
 [4 5 6]
 [7 8 9]]

D =
[[1 4 7]
 [2 5 8]
 [3 6 9]]

E =
[[1. 2. 3. 0. 0. 0.]
 [4. 5. 6. 0. 0. 0.]
 [7. 8. 9. 0. 0. 0.]
 [0. 0. 0. 1. 2. 3.]
 [0. 0. 0. 4. 5. 6.]
 [0. 0. 0. 7. 8. 9.]]

F =
[[ 14  32  50]
 [ 32  77 122]
 [ 50 122 194]]
```

If we have a matrix of numbers and we want to cleanly print this to the Python Shell, there are many options available. One possible approach is as follows (you'll understand why I am using this example in a few weeks). Suppose that we have a variable K1 that is a 6x6 matrix and has entries that are generally on the order of magnitude of 1 x 10^8.
If you use the command like above print(f"K1 = {K1}"), we get:

```
>>> print(f"K1 = {K1}")
K1 = [[ 1.5000e+08  0.0000e+00  0.0000e+00 -1.5000e+08  0.0000e+00  0.0000e+00]
 [ 0.0000e+00  1.0125e+07  6.7500e+06  0.0000e+00 -1.0125e+07  6.7500e+06]
 [ 0.0000e+00  6.7500e+06  6.0000e+06  0.0000e+00 -6.7500e+06  3.0000e+06]
 [-1.5000e+08  0.0000e+00  0.0000e+00  1.5000e+08  0.0000e+00  0.0000e+00]
 [ 0.0000e+00 -1.0125e+07 -6.7500e+06  0.0000e+00  1.0125e+07 -6.7500e+06]
 [ 0.0000e+00  6.7500e+06  3.0000e+06  0.0000e+00 -6.7500e+06  6.0000e+06]]
>>>
```

This format is not especially helpful or easy to read or interpret!

A simple alternative is:

print(f"K1 = 1 x 10^8 x \n{K1/1e8}\n") # the first \n command starts the variable display on a new ling so that the first row of the matrix is not offset.

which gives the following output, screenshot from the Python shell:

```
>>> print(f"K1 = 1 x 10^8 x \n{K1/1e8}")
K1 = 1 x 10^8 x
[[ 1.5      0.       0.      -1.5      0.       0.     ]
 [ 0.       0.10125  0.0675   0.      -0.10125  0.0675 ]
 [ 0.       0.0675   0.06     0.      -0.0675   0.03   ]
 [-1.5      0.       0.       1.5      0.       0.     ]
 [ 0.      -0.10125 -0.0675   0.       0.10125 -0.0675 ]
 [ 0.       0.0675   0.03     0.      -0.0675   0.06   ]]
```